

FPL Platform Implementation Blueprint

End-to-End Delivery Plan & Work Breakdown

Phase 1: Repository & Infrastructure Setup

- ☐ Initialize the Modular Monorepo with directories for `frontend/`, `backend/`, `data_pipeline/`, and `infrastructure/`. `Git`
- ☐ Define the foundational cloud infrastructure (Cloud SQL for PostgreSQL, Artifact Registry, Cloud Run) using declarative configurations. `Terraform`
- ☐ Configure CI/CD workflows to lint and selectively build containers based on directory changes. `GitHub Actions`
- ☐ Provision a managed PostgreSQL instance and execute initial schema creation scripts. `GCP`

Phase 2: Database Schema & Core Data Model

- ☐ Design the `managers` table to store league participants and team IDs. `PostgreSQL`
- ☐ Create the `gameweek_scores` table to track points, hits, and chip usage per week.
- ☐ Implement the `transfers` table to log players bought/sold, timestamps, and cost.
- ☐ Design a `pipeline_metadata` table to track the `finished` and `data_checked` status of each gameweek to prevent duplicate runs.

Phase 3: The Data Ingestion Engine

- ☐ Write the lightweight polling function to query `/bootstrap-static/` and validate if the current gameweek's `data_checked` flag is `True`. `Python`
- ☐ Deploy the polling function and schedule it to execute hourly. `Cloud Scheduler`
- ☐ Develop the main ETL batch job to pull data from `/entry/{id}/` and `/leagues-classic/{id}/standings/`, compute the rolling 3-game average, and load it into PostgreSQL.
- ☐ Containerize the ETL job to be triggered automatically upon successful polling validation. `Docker`

Phase 4: Backend API Services

- ☐ Initialize the API service to expose endpoints for the UI (e.g., `/api/v1/league/standings`, `/api/v1/league/transfers`). `FastAPI`
- ☐ Implement server-side logic to format custom metrics (rolling averages, form indicators).
- ☐ Write tests for data serialization and containerize the API. `Docker`
- ☐ Deploy the API as a serverless container, ensuring it scales to zero during the week to optimize costs. `Cloud Run`

Phase 5: The Mobile-Responsive UI (MVP)

- ☐ Scaffold the frontend framework and configure utility-first styling for rapid, responsive layouts. `Next.js` `Tailwind CSS`
- ☐ Build the 'News Feed' component to display a timeline of manager transfers and chip usage post-gameweek.
- ☐ Construct the custom Standings Table with integrated columns for rolling 3-game averages.
- ☐ Deploy the frontend to a public URL to facilitate easy sharing among league members.

Phase 6: Advanced Analytics & Expansion (Post-MVP)

These tasks are slated for subsequent iterations once the core pipeline is stable and the platform is live.

- ☐ Integrate visualization libraries to render the dynamic bar chart race for league positions.
- ☐ Construct the MLOps pipeline to extract historical underlying stats (xG, xA) and train a forecasting model. `MLflow` `LightGBM`
- ☐ Build the automated narrative generator to translate predicted points into match-up previews.